Switzerland's Export Destinations

Standard-HS4 export values across 30 selected partner areas, 2023-2025

Table of contents

1 Summary **2**

2 Requirements and configuration **2**

3 ELT process **3**
- 3.1 Database setup . . . 3
- 3.2 Extract and load discovery observations . . . 3
- 3.3 Extract HS4 detail . . . 3
- 3.4 Load HS4 observations . . . 3
- 3.5 Transform and check data quality . . . 4
- 3.6 Data structure . . . 8

4 Data analysis **10**
- 4.1 Partner values and annual shares . . . 10
- 4.2 Top ten HS4 headings in 2025 . . . 13
- 4.3 HS2 composition of the same five partner areas . . . 15

5 Conclusions **18**
- 5.1 Answers and practical relevance . . . 18

6 Learnings **19**

7 Sources **19**
- 7.1 Frozen data and result provenance . . . 19

1 Summary

This report studies Switzerland's annual goods exports to 30 selected partner areas during 2023-2025. These partner areas represent export destinations, including territories and statistical categories. The source uses original H6 classification, corresponding to HS 2022.

MongoDB selected the areas using cumulative all-commodity value, excluding World. The completed extraction returned 72,459 HS4 observations across 90 partner-year batches. The verified `swiss_trade_v2.trade_clean` collection contains 72,369 standard HS4 observations. The 90 nonstandard `9999` observations remain raw.

The report answers three questions:

1. Which partner areas lead in 2025, and how did their shares change since 2023?
2. Which ten HS4 headings have the highest 2025 export value across the selected areas?
3. How do the five largest 2025 partner areas differ in their HS2 composition?

The USA leads with 23.44% of selected 2025 value. Gold (7108) accounts for 37.09%. Chapter 71 accounts for 83.6% of the United Kingdom's selected value.

MongoDB performs the main calculations; Pandas and Matplotlib present the results. Shares describe the selected dataset, not all Swiss exports. Current-USD values do not establish physical-volume growth. Both partner selections use the comparison period retrospectively.

2 Requirements and configuration

The notebook uses `requests`, `pandas`, `matplotlib`, `python-dotenv`, `pymongo`, and `dnspython`. The local `requirements.txt` records Python dependencies. Quarto and LuaLaTeX render saved outputs without executing notebook cells.

The analysis uses `data/v2_hs4_snapshot_20261017T094752738553Z.json`, including its frozen H6 reference. Historical discovery uses `data/v2_partner_totals_20261016T143600945247Z.json`. The discovery snapshot contains 673 all-commodity `TOTAL` observations, not HS2 detail. The two snapshots serve different purposes.

Configuration uses `DATA_SOURCE`, `SNAPSHOT_PATH`, `HS4_DATA_SOURCE`, and `HS4_SNAPSHOT_PATH`. Live access uses the existing Comtrade API key configuration and `MONGODB_URI`. The notebook keeps credentials outside its source. Snapshot mode needs no API credentials or reference download. This PDF uses saved outputs, not a new API or database connection.

Setting	Scope
Reporter and flow	Switzerland (`757`), exports (`X`)
Period and frequency	2023-2025, annual goods
Classification	Original H6, HS 2022
Discovery	`partnerCode=None`, `cmdCode="TOTAL"`
HS4 detail	Fixed 30 partner areas, `cmdCode="AG4"`
Other dimensions	`partner2Code=0`, `customsCode="C00"`, `motCode=0`

Discovery omits the partner filter and retains World for context. MongoDB excludes World from partner selection.

3 ELT process

3.1 Database setup

PyMongo selects `swiss_trade_v2` through the existing Atlas connection. `partner_totals_raw` stores discovery evidence; `hs4_raw` stores HS4 source observations. `trade_clean` stores the verified embedded analytical model. The connection code remains in its original notebook position.

3.2 Extract and load discovery observations

ELT means Extract, Load, Transform. `requests.get()` retrieves annual JSON records; response-count checks detect incomplete responses. The notebook validates and freezes discovery records before loading them into MongoDB. MongoDB then selects partner areas before the separate HS4 extraction.

PyMongo's `MongoClient` uses `MONGODB_URI`; a separate `ping` checks connectivity. Discovery loading uses stable observation identifiers and replacement/upsert operations. Saved outputs establish 673 historical discovery observations and the completed partner selection.

3.2.1 Fixed partner-area selection

MongoDB ranks non-World partner areas by cumulative 2023-2025 all-commodity export value. The same 30 areas remain in scope for every year. Code `490`, `Other Asia, nes`, retains its official label. This retrospective selection does not represent worldwide coverage.

3.3 Extract HS4 detail

`AG4` requests HS4 headings, not four products. The completed extraction contains 90 partner-year batches across the fixed 30 areas. Validation checks response counts, original H6 classification, and aggregation level 4. The snapshot freezes partner selection, query settings, logs, reference codes, and control totals. Missing combinations remain absent.

The frozen H6 reference identifies standard HS4 headings; four digits alone do not establish membership. The raw collection also retains nonstandard source codes. Snapshot reproduction uses the saved reference without another download.

Exclusive file creation protects existing snapshots. The frozen HS4 snapshot preserves all source observations and extraction evidence. It contains no request headers or API keys.

The coverage audit compares source HS4 sums and standard-heading sums with frozen all-commodity control totals. Coverage differences do not justify rescaling values or inventing observations. Missing batches retain missing coverage; zero control totals have no defined percentage.

3.4 Load HS4 observations

The loader uses stable `_id` values and `ReplaceOne` upserts in batches of at most 1,000. Reruns replace matching observations without duplicating them. The loader rejects stale in-scope records instead of deleting them. Saved verification confirms 72,459 raw documents against the frozen snapshot.

The optional three-year top-50 drill-down contains 4,235 observed combinations. Its 265 absent combinations do not establish zero trade. The main clean collection retains all standard HS4 observations, not only this drill-down.

3.5 Transform and check data quality

Read-only MongoDB pipelines check the raw scope, standard-heading counts, and duplicate observation keys. `$type` distinguishes missing fields, explicit nulls, and numeric BSON types. Python repeats checks and displays MongoDB results. The following saved counts cover raw observations, including nonstandard `9999`.

```
if CHECK_HS4_DATA_QUALITY:
    stored_count = collection_quality.count_documents({})
    in_scope_count = collection_quality.count_documents(quality_raw_filter)
    print("Stored raw documents:", stored_count)
    print("Documents outside the selected source dimensions:", stored_count -
  in_scope_count)

    group_counts = {"$group": {
        "_id": {
            "period": "$period",
            "standardHeading": {"$in": ["$cmdCode", sorted(quality_heading_codes)]}
        },
        "observations": {"$sum": 1}
    }}
    sort_counts = {"$sort": {"_id.period": 1, "_id.standardHeading": -1}}
    pipeline = [quality_match, group_counts, sort_counts]
    quality_counts = list(collection_quality.aggregate(pipeline))
    print(pd.json_normalize(quality_counts).to_string(index=False))

    match_nonstandard = {"$match": {
        **quality_raw_filter, "cmdCode": {"$nin": sorted(quality_heading_codes)}
    }}
    group_codes = {"$group": {"_id": "$cmdCode", "observations": {"$sum": 1}}}
    pipeline = [match_nonstandard, group_codes, {"$sort": {"_id": 1}}]
    print("Nonstandard source codes:")
    pprint(list(collection_quality.aggregate(pipeline)))

    observation_fields = [
        "reporterCode", "period", "partnerCode", "partner2Code", "cmdCode",
        "classificationCode", "flowCode", "customsCode", "motCode"
    ]
    group_keys = {"$group": {
        "_id": {field: "$" + field for field in observation_fields},
        "documents": {"$sum": 1}
    }}
    match_duplicates = {"$match": {"documents": {"$gt": 1}}}
    pipeline = [quality_match, group_keys, match_duplicates, {"$count": "duplicateGroups"
  }]
    duplicate_result = list(collection_quality.aggregate(pipeline))
    print("Duplicate observation groups:", duplicate_result or [{"duplicateGroups": 0}])
```

```
Stored raw documents: 72459
Documents outside the selected source dimensions: 0
 observations _id.period  _id.standardHeading
        23805       2023                 True
           30       2023                False
        24188       2024                 True
           30       2024                False
        24376       2025                 True
           30       2025                False
Nonstandard source codes:
[{'_id': '9999', 'observations': 90}]
Duplicate observation groups: [{'duplicateGroups': 0}]
```

3.5.1 Embedded-model rationale

One document represents one year, reporter, destination area, and HS4 heading. Other reporting dimensions remain fixed. Related fields belong to small embedded objects, not one growing array per partner. This approach repeats labels but avoids routine lookups. Embedding alone does not guarantee faster aggregation.

HS4 codes have a fixed HS2 prefix: `0101` belongs to chapter `01`. Nested-set boundaries would add complexity without helping these queries.

The clean model retains `_id` and groups fields into `exportingCountry`, `destinationArea`, `tradeFlow`, `product`, `tradeMeasures`, and `dataQuality`. `exportingCountry` identifies the reporter, not production origin. `product.chapter.hsCode` supports HS2 grouping across all standard HS4 observations. The transformation reads `hs4_raw`, not discovery totals or the snapshot container.

Field group	Analytical purpose
`_id`, `year`	Trace raw observations and compare years
`exportingCountry`, `destinationArea`, `tradeFlow`	Identify reporter, partner area, and export direction
`product`	Identify HS4 headings and parent HS2 chapters
`tradeMeasures`	Retain current-USD values, net weights, quantities, and units
`dataQuality`	Retain reporting, classification, and estimation flags

For example, `partnerCode` becomes `destinationArea.comtradeCode`. `primaryValue` becomes `tradeMeasures.primaryValueUSD`. The model preserves copied values, leading-zero strings, nulls, units, and quality flags. It omits all-null `cifvalue` and all-zero `grossWgt`, which remain raw. The saved H6 reference supplies chapter labels after aggregation.

The course's Embedded Data Models (page 200) guides the document structure. Updating Collection Structure (page 209) guides `$addFields` and the separate `$out` save. Named dictionaries and pipeline lists follow the projection example (page 249) and Group And Push (page 251). The transformation needs no joins, separate entity collections, or type conversion.

3.5.2 Transformation pipeline

MongoDB filters standard observations, projects embedded objects, and derives each parent chapter. The read-only preview limits display to three documents without changing the full pipeline.

1. `match1` applies frozen reporting dimensions, selected areas, and the standard H6 heading list.
2. `project1` copies source values into embedded objects.
3. `add_fields1` derives the chapter after `product.hsCode` exists.

`$substrBytes` extracts two bytes from position zero. HS codes contain ASCII digits, so this expression preserves leading zeros. The pipeline contains no top-50 restriction or write stage.

```
if PREVIEW_TRADE_CLEAN:
    match1 = {"$match": quality_standard_filter}
    project1 = {"$project": {
        "_id": 1,
        "year": "$period",
        "exportingCountry": {
            "comtradeCode": "$reporterCode",
            "isoCode": "$reporterISO",
```

```
            "name": "$reporterDesc"
        },
        "destinationArea": {
            "comtradeCode": "$partnerCode",
            "isoCode": "$partnerISO",
            "name": "$partnerDesc"
        },
        "tradeFlow": {"code": "$flowCode", "description": "$flowDesc"},
        "product": {
            "classificationEdition": "$classificationCode",
            "aggregationLevel": "$aggrLevel",
            "hsCode": "$cmdCode",
            "description": "$cmdDesc"
        },
        "tradeMeasures": {
            "primaryValueUSD": "$primaryValue",
            "fobValueUSD": "$fobvalue",
            "netWeightKg": "$netWgt",
            "quantity": "$qty",
            "quantityUnitCode": "$qtyUnitCode",
            "quantityUnit": "$qtyUnitAbbr",
            "alternativeQuantity": "$altQty",
            "alternativeQuantityUnitCode": "$altQtyUnitCode",
            "alternativeQuantityUnit": "$altQtyUnitAbbr"
        },
        "dataQuality": {
            "isNetWeightEstimated": "$isNetWgtEstimated",
            "isQuantityEstimated": "$isQtyEstimated",
            "isAlternativeQuantityEstimated": "$isAltQtyEstimated",
            "isGrossWeightEstimated": "$isGrossWgtEstimated",
            "isOriginalClassification": "$isOriginalClassification",
            "isAggregate": "$isAggregate",
            "isLeaf": "$isLeaf",
            "isReported": "$isReported",
            "legacyEstimationFlag": "$legacyEstimationFlag"
        }
    }}
    add_fields1 = {"$addFields": {
        "product.chapter.hsCode": {"$substrBytes": ["$product.hsCode", 0, 2]}
    }}
    trade_clean_pipeline = [match1, project1, add_fields1]
```

```
if PREVIEW_TRADE_CLEAN:
    pipeline = trade_clean_pipeline + [{"$limit": 3}]
    preview_documents = list(collection.aggregate(pipeline))
    pprint(preview_documents)
```

Read-only checks cover the full transformation, not just the three preview documents. They compare every copied field and BSON type, observation keys, chapter prefixes, and monetary sums. The checks preserve source nulls, numeric zeros, units, and flags. Primary and FOB sums undergo independent comparisons.

3.5.3 Save and verify the clean collection

The save cell uses an independent copy of the checked pipeline and rejects later stage changes. It refuses any existing destination because `$out` can replace collections. The successful save used the full pipeline, not the limited preview. Current settings retain `SAVE_TRADE_CLEAN = False`. The existing verified destination requires no new write.

Saved outputs confirm these `swiss_trade_v2.trade_clean` counts:

Year	Standard HS4 observations
2023	23,805
2024	24,188
2025	24,376
Total	72,369

Full saved-result checks cover identifiers, duplicate keys, types, null/zero counts, and primary and FOB sums. Full-document equality checks cover the three preview documents, not every saved document. The raw helper confirms 72,459 unchanged observations against the frozen snapshot.

Measurement	Raw numeric zeros	Clean numeric zeros
Net weight	98	8
Quantity	9,531	9,441
Alternative quantity	43,029	42,939

Both scopes contain 495 null net weights. Raw gross weight contains only numeric zeros; the clean model omits this measurement. Different zero counts reflect the 90 excluded `9999` observations, not measurement replacement. Nulls and numeric zeros do not establish confirmed measured zero trade.

3.6 Data structure

The schema excerpts compare flat raw fields with embedded clean objects. Gaps mark omitted fields; the UML shows the complete stored model.

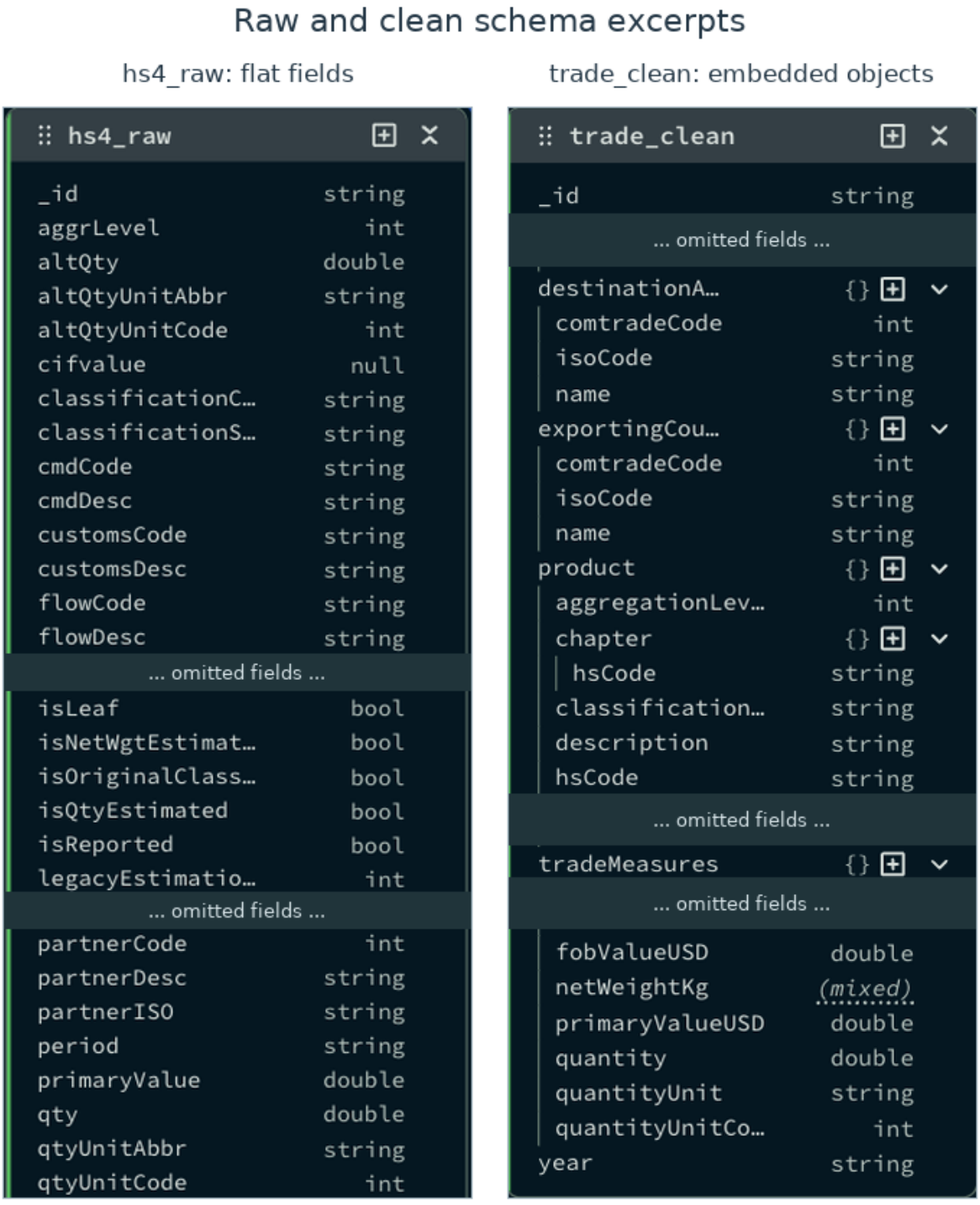


Figure 1: Selected raw and clean schema fields.

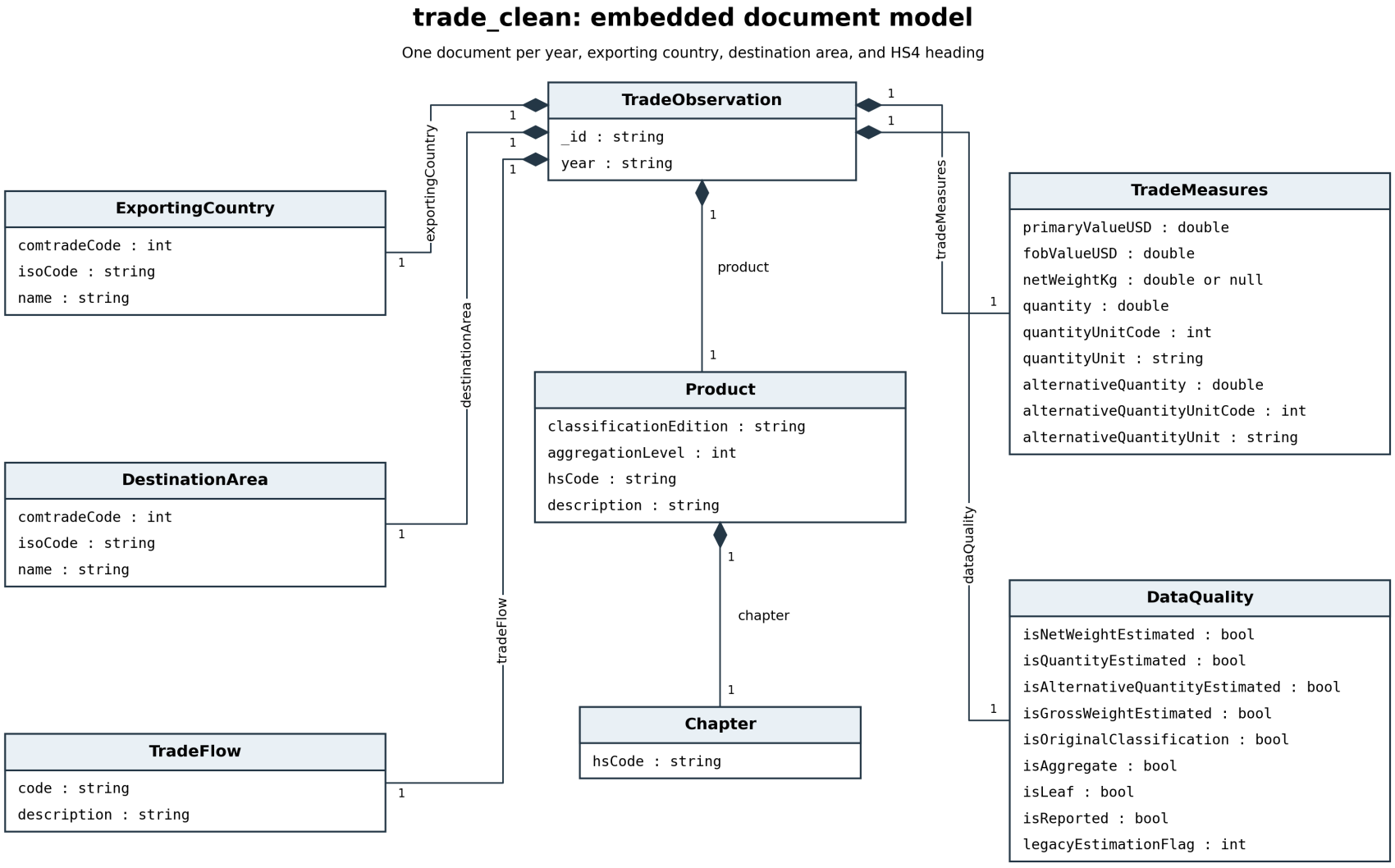


Figure 2: Complete UML of the verified embedded model.

4 Data analysis

The analyses move from partner areas to HS4 headings, then compare HS2 composition. All main calculations use MongoDB; Pandas formats tables and Matplotlib draws charts.

4.1 Partner values and annual shares

Question: Which selected partner areas lead in 2025, and how did their shares change since 2023?

Purpose: Compare destination importance while separating shares from absolute export value.

Method: Calculate annual values from all standard HS4 observations across the fixed 30 areas. Select the five largest 2025 areas only after calculating denominators. Keep these five areas across all comparisons.

4.1.1 Annual denominators

The first `$group` sums export value and counts HS4 observations per partner area and year. The second `$group` calculates annual totals and uses `$push` to retain partner rows. `$unwind` restores those rows; `$project`, `$divide`, and `$multiply` calculate percentage shares. `$first` retains a consistent destination label, not a highest-value destination. These temporary arrays do not change the stored model. Annual totals repeat after `$unwind`; summing them would double-count value.

Result: The saved annual table covers all 30 selected partner areas. Observation counts describe HS4 records, not shipments.

```
if ANALYZE_PARTNERS:
    partner_shares_checked = False
    match_partner_years = {"$match": partner_analysis_filter}
    group_partner_year = {"$group": {
        "_id": {"year": "$year", "destinationCode": "$destinationArea.comtradeCode"},
        "destinationName": {"$first": "$destinationArea.name"},
        "valueUSD": {"$sum": "$tradeMeasures.primaryValueUSD"},
        "observations": {"$sum": 1}
    }}
    group_annual_totals = {"$group": {
        "_id": "$_id.year",
        "annualValueUSD": {"$sum": "$valueUSD"},
        "partnerAreas": {"$sum": 1},
        "observations": {"$sum": "$observations"},
        "destinations": {"$push": {
            "destinationCode": "$_id.destinationCode",
            "destinationName": "$destinationName",
            "valueUSD": "$valueUSD", "observations": "$observations"
        }}
    }}
    project_annual_scope = {"$project": {
        "_id": 0, "year": "$_id", "annualValueUSD": 1,
        "partnerAreas": 1, "observations": 1
    }}
    unwind_destinations = {"$unwind": "$destinations"}
    project_partner_shares = {"$project": {
        "_id": 0, "year": "$_id",
        "destinationCode": "$destinations.destinationCode",
        "destinationName": "$destinations.destinationName",
        "valueUSD": "$destinations.valueUSD",
        "observations": "$destinations.observations",
        "annualValueUSD": 1,
```

```
        "sharePercent": {"$multiply": [
            {"$divide": ["$destinations.valueUSD", "$annualValueUSD"]}, 100
        ]}
    }}
    sort_partner_shares = {"$sort": {"year": 1, "valueUSD": -1, "destinationCode": 1}}
    partner_share_pipeline = [
        match_partner_years, group_partner_year, group_annual_totals,
        unwind_destinations, project_partner_shares, sort_partner_shares
    ]
```

```
year  annualValueUSD  partnerAreas  observations
2023 388,003,699,587            30         23805
2024 409,652,921,569            30         24188
2025 516,753,110,733            30         24376
Annual shares across the full selected dataset sum to 100%.
```

4.1.2 Fixed five-area comparison

The 2025 filter, descending sort, and limit follow the full-denominator calculation. Comtrade codes break value ties consistently. The same five codes support the later HS2 composition analysis.

```
if ANALYZE_PARTNERS:
    if not partner_shares_checked:
        raise ValueError("Run the annual share checks before selecting destinations.")
    match_rank_year = {"$match": {"year": PARTNER_RANK_YEAR}}
    sort_rank_year = {"$sort": {"valueUSD": -1, "destinationCode": 1}}
    limit_top5_partners = {"$limit": 5}
    pipeline = partner_share_pipeline + [match_rank_year, sort_rank_year,
  limit_top5_partners]
    top5_partners_2025 = list(collection_clean.aggregate(pipeline))
    if len(top5_partners_2025) != 5:
        raise ValueError("Expected five observed destinations in the 2025 ranking.")
    top5_partner_codes = [row["destinationCode"] for row in top5_partners_2025]
```

`$cond` selects endpoint years and returns null otherwise. `$max` retrieves each selected endpoint, not the largest annual share. `$subtract` calculates 2025-minus-2023 percentage-point changes. A change from 10% to 12% equals two percentage points, not 2% growth. Missing endpoints retain null changes instead of artificial zeros. `yearsObserved` counts available annual results.

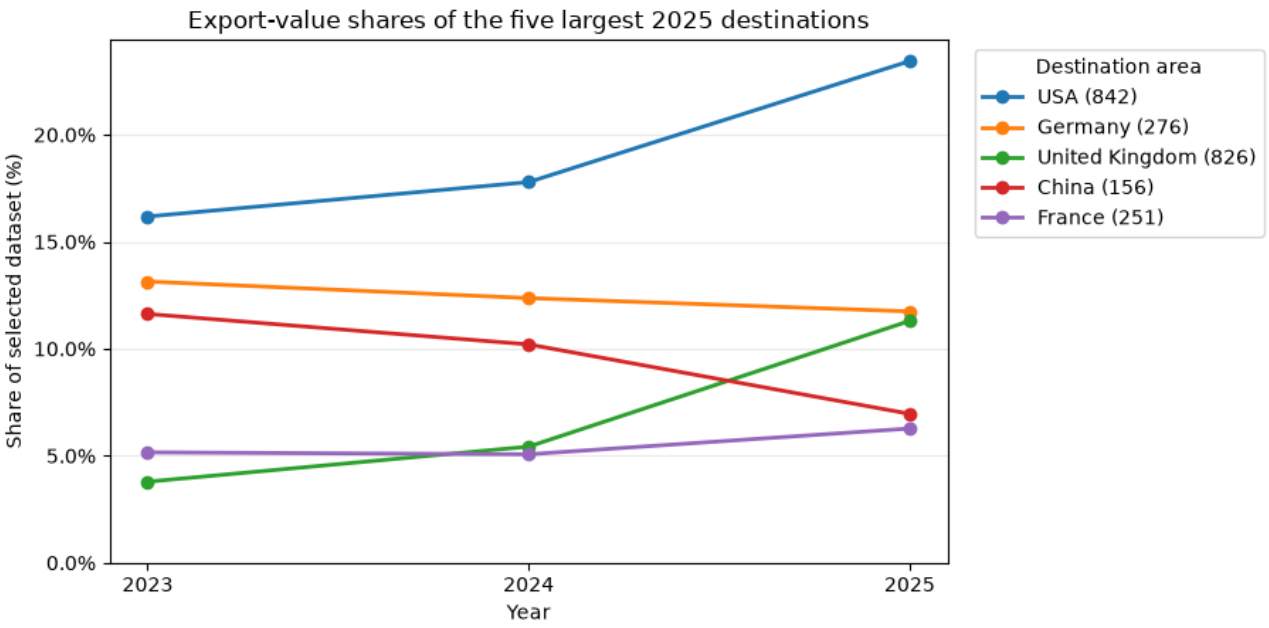
Result: The table shows current-USD values, annual shares, and percentage-point changes for the fixed five areas.

```
if ANALYZE_PARTNERS:
    match_top5_partners = {"$match": {"destinationCode": {"$in": top5_partner_codes}}}
    group_partner_changes = {"$group": {
        "_id": "$destinationCode",
        "destinationName": {"$first": "$destinationName"},
        "yearsObserved": {"$sum": 1},
        "value2025USD": {"$max": {"$cond": [
            {"$eq": ["$year", "2025"]}, "$valueUSD", None
        ]}},
        "share2023Pct": {"$max": {"$cond": [
            {"$eq": ["$year", "2023"]}, "$sharePercent", None
        ]}},
        "share2025Pct": {"$max": {"$cond": [
            {"$eq": ["$year", "2025"]}, "$sharePercent", None
        ]}}
    }}
    project_partner_changes = {"$project": {
```

```
        "_id": 0, "destinationCode": "$_id", "destinationName": 1,
        "value2025USD": 1, "share2025Pct": 1, "yearsObserved": 1,
        "shareChangePP": {"$cond": [
            {"$and": [{"$ne": ["$share2023Pct", None]}, {"$ne": ["$share2025Pct", None
  ]}]},
            {"$subtract": ["$share2025Pct", "$share2023Pct"]},
            None
        ]}
    }}
    sort_partner_changes = {"$sort": {"value2025USD": -1, "destinationCode": 1}}
    pipeline = partner_share_pipeline + [
        match_top5_partners, group_partner_changes,
        project_partner_changes, sort_partner_changes
    ]
    partner_change_rows = list(collection_clean.aggregate(pipeline))
    if len(partner_change_rows) != 5 or {row["destinationCode"] for row in
  partner_change_rows} != set(top5_partner_codes):
        raise ValueError("The overview must retain the same five selected destinations.")
    partner_overview = pd.DataFrame(partner_change_rows)[[
        "destinationName", "value2025USD", "share2025Pct", "shareChangePP", "
  yearsObserved"
    ]]
    partner_overview_display = partner_overview.rename(columns={
        "destinationName": "Destination area", "value2025USD": "2025 value (USD)",
        "share2025Pct": "2025 share (%)", "shareChangePP": "2023-25 change (pp)",
        "yearsObserved": "Years covered"
    })
    print(partner_overview_display.to_string(index=False, na_rep="N/A", formatters={
        "2025 value (USD)": "{:,.0f}".format,
        "2025 share (%)": "{:.2f}".format,
        "2023-25 change (pp)": "{:+.2f}".format
    }))
```

```
destinationName    value2025USD share2025Pct shareChangePP  yearsObserved
            USA 121,135,599,115        23.44         +7.26              3
        Germany  60,694,234,233        11.75         -1.40              3
 United Kingdom  58,400,701,852        11.30         +7.51              3
          China  35,953,182,608         6.96         -4.67              3
         France  32,409,327,180         6.27         +1.11              3
```

The chart follows the same five areas through 2023-2025. Their shares need not sum to 100%. Missing observations leave gaps, not artificial zeros.



Shares use all 30 selected partner areas, not only the five displayed areas.
Gaps indicate absent observations, not measured zero trade.

Interpretation: Selected export value rises from USD 388.0 billion in 2023 to USD 516.8 billion in 2025. The USA leads in 2025 with USD 121.1 billion and a 23.44% share. The United Kingdom gains 7.51 percentage points, the largest gain among the displayed five. China loses 4.67 points; Germany loses 1.40 points.

Limitation: Share changes do not necessarily indicate changes in absolute value. Retrospective selection limits generalization. Current-USD changes can reflect prices, exchange rates, quantities, and product mix. These comparisons establish neither physical-volume growth nor causation.

4.2 Top ten HS4 headings in 2025

Question: Which HS4 headings have the highest 2025 export value across the 30 selected partner areas?

Purpose: Identify headings that dominate nominal export value.

Method: Rank all standard HS4 observations in 2025, not the earlier top-50 drill-down. Calculate the full annual denominator before selecting ten headings.

MongoDB first calculates the annual total with `$group` and `$sum`. It then groups by HS4 code, sorts by value, breaks ties by code, and limits display to ten. `$first` retains each heading's consistent source description. `$project` calculates USD billions and shares using the complete annual denominator. Python passes that MongoDB total into the ranking without recalculating it.

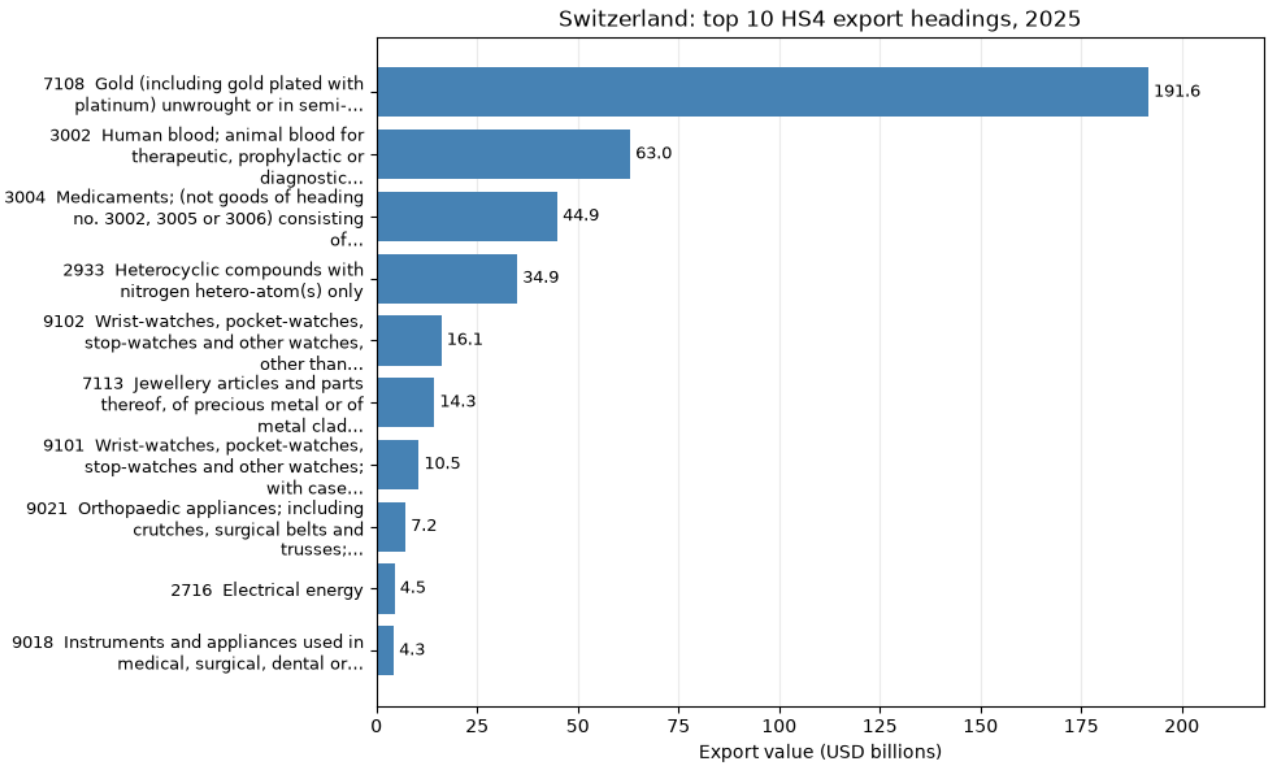
Result: The saved table and chart shorten labels, not codes or calculated values.

```
if ANALYZE_HS4_2025:
    from textwrap import shorten

    hs4_2025_checked = False
    group_product_year_total = {"$group": {
        "_id": None, "totalValueUSD": {"$sum": "$tradeMeasures.primaryValueUSD"},
        "observations": {"$sum": 1}
    }}
    pipeline = [match_products_2025, group_product_year_total]
    product_year_totals = list(collection_clean.aggregate(pipeline))
```

```
    if (not product_year_totals or product_year_totals[0]["observations"] != 24376
            or not math.isfinite(product_year_totals[0]["totalValueUSD"])
            or product_year_totals[0]["totalValueUSD"] <= 0):
        raise ValueError("Check the positive full-year total and 24,376 observations
  before ranking products.")
    product_2025_total_usd = product_year_totals[0]["totalValueUSD"]
    group_hs4_2025 = {"$group": {
        "_id": "$product.hsCode", "description": {"$first": "$product.description"},
        "valueUSD": {"$sum": "$tradeMeasures.primaryValueUSD"},
        "observations": {"$sum": 1}
    }}
    sort_hs4_2025 = {"$sort": {"valueUSD": -1, "_id": 1}}
    limit_top10_hs4 = {"$limit": 10}
    project_hs4_2025 = {"$project": {
        "_id": 0, "hsCode": "$_id", "description": 1, "valueUSD": 1, "observations": 1,
        "valueUSDbn": {"$divide": ["$valueUSD", 1e9]},
        "datasetSharePct": {"$multiply": [{"$divide": ["$valueUSD",
  product_2025_total_usd]}, 100]}
    }}
    pipeline = [match_products_2025, group_hs4_2025, sort_hs4_2025, limit_top10_hs4,
  project_hs4_2025]
    top10_hs4_rows = list(collection_clean.aggregate(pipeline))
    if len(top10_hs4_rows) != 10 or any(row["hsCode"] not in quality_heading_codes for
  row in top10_hs4_rows):
        raise ValueError("Expected ten standard HS4 headings from the full 2025 dataset."
  )
    top10_hs4 = pd.DataFrame(top10_hs4_rows)
    top10_hs4_display = top10_hs4[["hsCode", "description", "valueUSDbn", "
  datasetSharePct"]].copy()
    top10_hs4_display["description"] = [shorten(text, width=36, placeholder="…") for text
   in top10_hs4_display["description"]]
    top10_hs4_display = top10_hs4_display.rename(columns={
        "hsCode": "HS4", "description": "Product (short label)",
        "valueUSDbn": "Value (USD bn)", "datasetSharePct": "Dataset share (%)"
    })
    print(top10_hs4_display.to_string(index=False, formatters={
        "Value (USD bn)": "{:.2f}".format, "Dataset share (%)": "{:.2f}".format
    }))
    hs4_2025_checked = True
```

```
 HS4               Product (short label) Value (USD bn) Dataset share (%)
7108   Gold (including gold plated with…         191.64             37.09
3002      Human blood; animal blood for…          62.99             12.19
3004 Medicaments; (not goods of heading…          44.91              8.69
2933        Heterocyclic compounds with…          34.94              6.76
9102     Wrist-watches, pocket-watches,…          16.09              3.11
7113       Jewellery articles and parts…          14.35              2.78
9101     Wrist-watches, pocket-watches,…          10.51              2.03
9021  Orthopaedic appliances; including…           7.21              1.40
2716                   Electrical energy           4.48              0.87
9018 Instruments and appliances used in…           4.29              0.83
```



Current USD; standard HS4 observations across all 30 selected partner areas.
Labels are shortened for display. Values do not represent all Swiss exports.

Interpretation: Gold (7108) leads with USD 191.64 billion and 37.09% of selected 2025 value. Heading 3002 follows at 12.19%; medicaments under 3004 account for 8.69%. Heading 3002 includes blood, immunological products, vaccines, and related goods. Heading 9101 covers precious-metal or precious-metal-clad watch cases; 9102 covers other watches.

Limitation: The top ten need not sum to 100%; other headings remain in the denominator. Value rankings do not establish Swiss production origin, profitability, or physical-volume growth.

4.3 HS2 composition of the same five partner areas

Question: How do the five largest 2025 partner areas differ in their HS2 product composition?

Purpose: Compare broad product mixes using common chapter categories.

Method: Group all standard HS4 observations by `product.chapter.hsCode`. Choose five chapters by combined value across the same five partner areas. Do not restrict totals to the top-ten or top-50 headings. The saved H6 reference supplies labels after MongoDB aggregation without another download.

```
if ANALYZE_HS2_COMPOSITION:
    composition_2025_checked = False
    leading_chapter_codes = []
    composition_filter = product_2025_filter.copy()
    composition_filter["destinationArea.comtradeCode"] = {"$in":
  composition_partner_codes}
    match_composition_2025 = {"$match": composition_filter}
    group_partner_chapters = {"$group": {
        "_id": {"destinationCode": "$destinationArea.comtradeCode", "chapterCode": "
  $product.chapter.hsCode"},
        "destinationName": {"$first": "$destinationArea.name"},
        "valueUSD": {"$sum": "$tradeMeasures.primaryValueUSD"}, "observations": {"$sum":
  1}
```

```
    }}
    composition_base_pipeline = [match_composition_2025, group_partner_chapters]
    group_leading_chapters = {"$group": {"_id": "$_id.chapterCode", "valueUSD": {"$sum":
  "$valueUSD"}}}
    sort_leading_chapters = {"$sort": {"valueUSD": -1, "_id": 1}}
    limit_leading_chapters = {"$limit": 5}
    pipeline = composition_base_pipeline + [group_leading_chapters, sort_leading_chapters
  , limit_leading_chapters]
    leading_chapter_rows = list(collection_clean.aggregate(pipeline))
    if len(leading_chapter_rows) != 5 or any(row["_id"] not in chapter_names for row in
  leading_chapter_rows):
        raise ValueError("Expected five HS2 chapters with labels in the saved H6
  reference.")
    leading_chapter_codes = [row["_id"] for row in leading_chapter_rows]
```

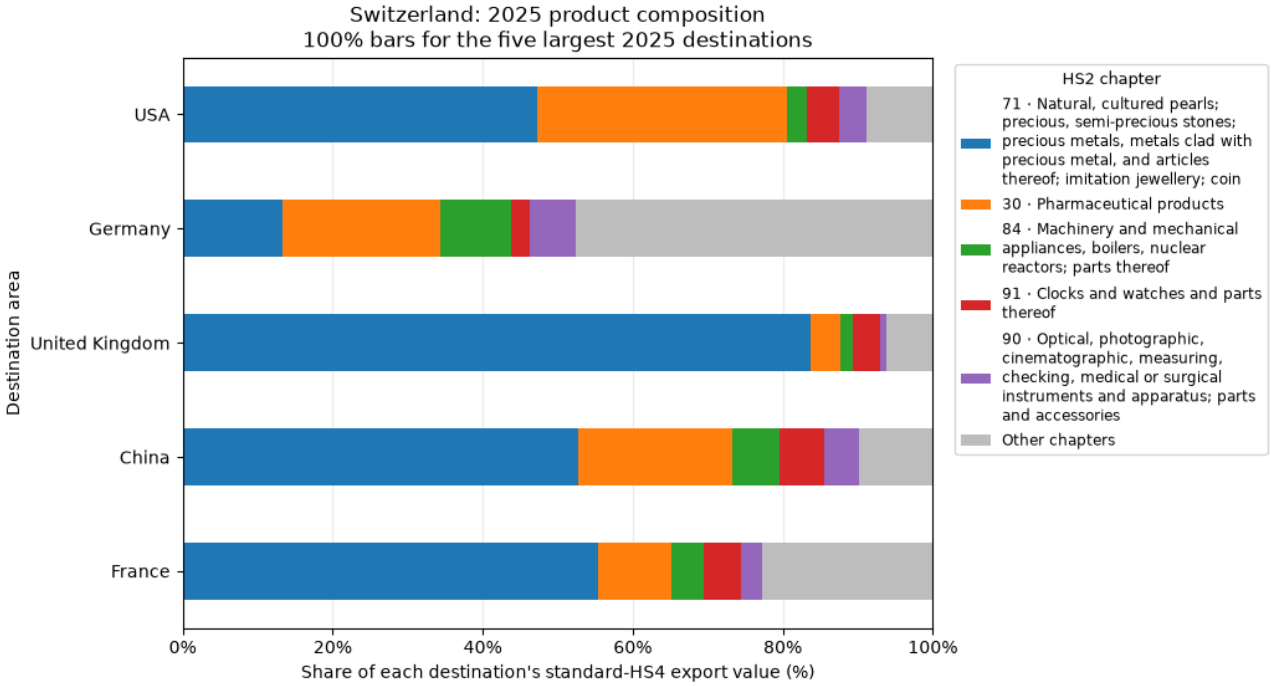
`$cond` assigns observed chapters outside the common five to Other. Further `$group` stages calculate category values and each partner area's full denominator. `$push` and `$unwind` retain category rows beside their partner total. That total repeats across categories; summing it would double-count value.

Result: Full-precision category shares total 100% for each area. Values and observation counts match the first analysis's 2025 results. A dash means no recorded category observation, not measured zero trade. Other combines observed values; it does not create absent observations.

```
if ANALYZE_HS2_COMPOSITION:
    composition_2025_checked = False
    if len(leading_chapter_codes) != 5:
        raise ValueError("Run the common-chapter selection before calculating composition
  .")
    group_display_categories = {"$group": {
        "_id": {
            "destinationCode": "$_id.destinationCode",
            "categoryCode": {"$cond": [
                {"$in": ["$_id.chapterCode", leading_chapter_codes]}, "$_id.chapterCode",
   "Other"
            ]}
        },
        "destinationName": {"$first": "$destinationName"},
        "valueUSD": {"$sum": "$valueUSD"}, "observations": {"$sum": "$observations"}
    }}
    group_composition_totals = {"$group": {
        "_id": "$_id.destinationCode", "destinationName": {"$first": "$destinationName"},
        "partnerValueUSD": {"$sum": "$valueUSD"},
        "categories": {"$push": {
            "categoryCode": "$_id.categoryCode", "valueUSD": "$valueUSD", "observations":
   "$observations"
        }}
    }}
    unwind_composition_categories = {"$unwind": "$categories"}
    project_composition_shares = {"$project": {
        "_id": 0, "destinationCode": "$_id", "destinationName": 1, "partnerValueUSD": 1,
        "categoryCode": "$categories.categoryCode", "valueUSD": "$categories.valueUSD",
        "observations": "$categories.observations",
        "sharePercent": {"$multiply": [{"$divide": ["$categories.valueUSD", "
  $partnerValueUSD"]}, 100]}
    }}
    sort_composition_shares = {"$sort": {"destinationCode": 1, "categoryCode": 1}}
    composition_share_pipeline = composition_base_pipeline + [
        group_display_categories, group_composition_totals,
        unwind_composition_categories, project_composition_shares,
  sort_composition_shares
    ]
```

```
    pipeline = composition_share_pipeline
    composition_share_rows = list(collection_clean.aggregate(pipeline))
    group_composition_checks = {"$group": {
        "_id": "$destinationCode", "partnerValueUSD": {"$first": "$partnerValueUSD"},
        "sharePercentTotal": {"$sum": "$sharePercent"}, "observations": {"$sum": "
  $observations"}
    }}
    pipeline = composition_share_pipeline + [group_composition_checks, {"$sort": {"_id":
  1}}]
    composition_scope_rows = list(collection_clean.aggregate(pipeline))
    if {row["_id"] for row in composition_scope_rows} != set(composition_partner_codes):
        raise ValueError("The composition must retain the original five destinations.")
    expected_partner_totals = {row["destinationCode"]: row for row in top5_partners_2025}
    for row in composition_scope_rows:
        expected = expected_partner_totals[row["_id"]]
        if (not math.isclose(row["sharePercentTotal"], 100, abs_tol=1e-8)
                or not math.isclose(row["partnerValueUSD"], expected["valueUSD"], rel_tol
  =1e-12, abs_tol=0.01)
                or row["observations"] != expected["observations"]):
            raise ValueError("Composition shares, totals, or counts differ from the first
   analysis.")
    composition_share_table = pd.DataFrame(composition_share_rows)
    composition_category_codes = leading_chapter_codes + ["Other"]
    composition_matrix = composition_share_table.pivot(
        index="destinationCode", columns="categoryCode", values="sharePercent"
    ).reindex(index=composition_partner_codes, columns=composition_category_codes)
    composition_destination_names = {row["destinationCode"]: row["destinationName"] for
  row in top5_partners_2025}
    composition_display = composition_matrix.rename(
        index=composition_destination_names,
        columns={code: ("Other" if code == "Other" else "HS2 " + code) for code in
  composition_category_codes}
    )
    composition_display.index.name = "Destination area"
    composition_display.columns.name = "Share (%)"
    print(composition_display.to_string(na_rep="-", float_format="{:.1f}".format))
    print("Category shares total 100% and partner values/counts match the first analysis.
  ")
    composition_2025_checked = True
```

```
Share (%)         HS2 71  HS2 30  HS2 84  HS2 91  HS2 90  Other
Destination area
USA                 47.3    33.3     2.6     4.3     3.7    8.9
Germany             13.3    21.1     9.4     2.4     6.2   47.6
United Kingdom      83.6     4.1     1.6     3.6     0.9    6.2
China               52.7    20.6     6.2     6.1     4.6    9.9
France              55.5     9.7     4.4     5.0     2.7   22.7
Category shares total 100% and partner values/counts match the first analysis.
```



HS2 shares of standard-HS4 value for the same five leaders within the 30 selected partner areas.
Not all Swiss exports. Missing segments mean no recorded category observations, not measured zero trade.

Interpretation: Chapter 71 accounts for 83.6% of the United Kingdom's selected value. This chapter includes precious metals, stones, and jewellery, not gold alone. Pharmaceutical products (30) account for 33.3% of the USA's mix and 21.1% of Germany's. Germany has the largest Other share, at 47.6%.

Limitation: Other hides chapter detail and does not measure diversification. Equal bar lengths compare percentages, not equal export values. Rounding can produce displayed totals of 99.9% or 100.1%; checks use full precision. Never add HS2 totals to their child-heading values.

5 Conclusions

5.1 Answers and practical relevance

1. The USA leads selected 2025 value with 23.44%. Germany, the United Kingdom, China, and France complete the five largest partner areas. The United Kingdom gains the most share among these five: 7.51 percentage points since 2023. The USA gains 7.26 points; China loses 4.67 points. These comparisons identify areas for further market research, not predictions of demand or country risk.
2. Gold (7108) leads the HS4 ranking with 37.09%, followed by 3002 at 12.19% and 3004 at 8.69%. High nominal values identify product groups for investigation, not shipment volumes or profitability.
3. The five areas have different chapter mixes. Chapter 71 dominates the United Kingdom; pharmaceuticals contribute substantially to the USA and Germany. Germany's 47.6% Other share leaves more chapter detail outside the five common categories. These comparisons guide follow-up questions, not commercial recommendations or explanations of earlier share changes.

6 Learnings

1. **Modeling around queries.** I learned to choose embedding or references from the queries I need. Embedding keeps related details together but repeats labels; references separate entities and may require joins. Here, one trade observation embeds partner-area, product, measurement, and quality details.
2. **Identifiers and collections.** I learned that `_id` uniquely identifies a document within its collection. An HS4 code alone cannot identify an observation across different years and partner areas. Stable identifiers let upserts replace matching observations instead of creating duplicates.
3. **Flexible schemas and BSON types.** MongoDB's flexible schema does not remove the need for consistent fields and BSON types. I kept HS codes as strings to preserve leading zeros. I used `$type` to distinguish missing fields, nulls, and numeric values.
4. **Aggregation stages and order.** I learned to build aggregations as ordered stages. `$match` filters observations, `$project` shapes documents, and `$group` calculates summaries. I calculated full denominators before `$sort` and `$limit` selected display rows.
5. **Grouping and arrays.** I learned that grouping and unwinding change what one pipeline document represents. `$push` collects results into arrays; `$unwind` returns one document per array element. I avoided summing repeated annual totals after unwinding partner results.
6. **Transformations and database writes.** I learned that transforming query results differs from changing stored data. `$project` and `$addFields` reshape pipeline results; `$out` writes them and can replace an existing collection. I kept raw evidence separate, checked the destination, and verified the saved collection.

7 Sources

- UN Comtrade, API documentation: the source API for annual Swiss export observations and query parameters.
- UN Comtrade, H6 classification reference and partner-area reference: heading membership, partner codes, and official labels.
- MongoDB, Data Modeling: document structure, embedded objects, and references.
- MongoDB, aggregation stages, `$out`, and `$substrBytes`: pipeline operations, collection writes, and chapter-prefix derivation.
- MongoDB, PyMongo aggregation guide: executing MongoDB aggregation pipelines from Python.
- UN Comtrade, Data Use and Re-dissemination: conditions to check before sharing source observations.

7.1 Frozen data and result provenance

The analytical input is `data/v2_hs4_snapshot_20261017T094752738553Z.json`, including HS4 observations, the H6 reference, partner selection, and extraction logs. Historical discovery uses `data/v2_partner_totals_20261016T143600945247Z.json`. This separate snapshot contains all-commodity `TOTAL` observations, not HS2 product detail.

The saved MongoDB outputs in this notebook support the reported counts, rankings, shares, and verification results. UN Comtrade may revise data after the frozen extraction.